

CricGeek Complete Technical Documentation

Developer handoff and system documentation compiled from the repository

Compiled from the repository and supporting markdown documentation.

Prepared for: developer handoff and technical onboarding

Prepared: 2026-09-20 12:55 UTC

Document scope	Repository-backed, full technical system documentation
Primary stack	Next.js 16, React 19, TypeScript, Prisma, SQL Server, NextAuth, Python AI services
Core concerns	Match data, writer system, AI scoring, contests, commentary, auth, fact-checking
Primary runtime	Node.js app runtime; AI tooling and Python services are external dependencies

Table of contents

1. Executive overview
2. Architecture
3. Project structure
4. Frontend architecture
5. Backend architecture
6. Database and schema
7. Authentication and security
8. API reference
9. Pages
10. Features
11. EQS
12. Voice-to-Commentary
13. External services
14. Configuration
15. Dependencies
16. Implementation status
17. Known issues
18. Developer handoff

README

CricGeek Technical Documentation

This documentation package describes the CricGeek codebase as it exists in this repository at the time of review. It intentionally documents only what is supported by the code and highlights where the implementation is partial, planned, or unused.

Documentation set

- 01-ARCHITECTURE.md — Architecture overview and system diagrams
- 02-PROJECT-STRUCTURE.md — Real directory and module map
- 03-FRONTEND.md — Frontend framework, pages, components, UI flows
- 04-BACKEND.md — Next.js app router backend and server-side services
- 05-DATABASE.md — Prisma schema and database architecture
- 06-AUTHENTICATION-SECURITY.md — Auth, roles, security posture, env controls
- 07-API-REFERENCE.md — Route-by-route API documentation
- 08-PAGES.md — Page-by-page feature inventory
- 09-FEATURES.md — Major features and business logic
- 10-EQS.md — EQS technical implementation and status
- 11-VOICE-TO-COMMENTARY.md — Voice commentary system status and implementation
- 12-EXTERNAL-SERVICES.md — External APIs and provider usage
- 13-CONFIGURATION.md — Environment variables and operational configuration
- 14-DEPENDENCIES.md — Runtime, dev, and AI dependencies
- 15-IMPLEMENTATION-STATUS.md — Feature matrix and implementation status
- 16-KNOWN-ISSUES.md — Confirmed issues and risks
- 17-DEVELOPER-HANDOVER.md — Handover guidance for a new developer

Executive summary

CricGeek is a Next.js application focused on cricket content, live match intelligence, writer/community features, AI-assisted scoring, and commentary workflows. The frontend is implemented primarily in the App Router under `src/app`, with reusable UI in `src/components`. Server logic and route handlers live in the same app folder as route modules and shared service code in `src/lib`.

The system depends on:

- Next.js 16 + React 19 for the frontend and App Router API layer
- Prisma ORM with SQL Server datasource in `schema.prisma`

- SQLite-style local dev examples and MySQL-like examples in environment files, which indicates configuration drift
- Ollama for local model inference with Qwen defaults
- SportMonks for live cricket data
- optional Deepgram and external search providers for transcription and fact checking
- Python AI services under `ai_service` and `insights_service` for additional analysis and scoring features

Important status notice

This repository contains a large amount of functionality, but several features are assembled as partial or environment-dependent services. Some features are clearly implemented, some are behind environment variables, and some are intentionally staged as demo-mode or planned extension work.

The docs below distinguish between:

- **IMPLEMENTED:** directly supported by repository code
- **PARTIALLY IMPLEMENTED:** code exists but is incomplete or relies on external services or demo fallbacks
- **PLANNED / NOT IMPLEMENTED:** referenced in architecture or docs, but no supporting code path was found
- **IMPLEMENTED BUT CURRENTLY UNUSED:** present in code but not actively wired into the main path

Core product areas

- Live match and calendar information
- Writer community with blog publishing and social interactions
- EQS-style writing analysis and scoring
- Commentary publishing and transcription workflows
- Contest and leaderboard mechanics
- Historical match analysis / EDA-related features
- AI-assisted fact checking and originality checks

Primary runtime entry points

- Next.js app: `package.json` and `src/app`
- Prisma schema: `prisma/schema.prisma`
- Python AI services: `ai_service` and `insights_service`
- Supporting scripts: `scripts` and `docs`

Reading order for a new engineer

1. Architecture overview 2. Route and data flow 3. Database model 4. Auth + security 5. API layer 6. Feature logic and EQS 7. Known issues and environment gaps

Scope boundaries

This package does not modify application code or infrastructure. It reflects the repository as-is and documents known risks and gaps, especially where the environment or external services are required for full runtime behavior.

ARCHITECTURE

01-ARCHITECTURE

System overview

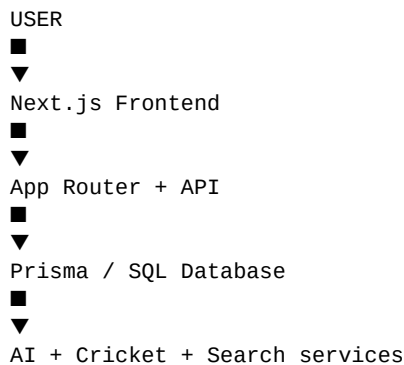
CricGeek is a full-stack cricket platform with a Node/Next.js application and auxiliary Python services. The main user-facing application runs through the Next.js App Router in `src/app`. The app consumes live cricket data, handles blog and commentary flows, and integrates with language-model-based quality and analysis services.

The architecture is best understood as:

USER ↓ Next.js frontend (App Router) ↓ Route handlers and server actions ↓ Prisma + database ↓ External cricket and AI providers ↓ Python AI services for advanced features

High-level architecture diagram

System diagram (source Mermaid diagram preserved in markdown):



flowchart TD
 U[User / Writer / Admin] --> FE[Next.js Frontend\nApp Router + React Components]
 FE --> API[Next.js API Routes\nsrc/app/api]
 API --> LIB[Shared Logic\nsrc/lib]
 LIB --> DB[(Prisma / SQL Database)]
 DB --> SM[SportMonks API]
 LIB --> OLLAMA[Ollama Qwen model]
 LIB --> DEEP[Deepgram STT]
 LIB --> NEWS[GNews / TheNewsAPI]
 LIB --> RAG[RAG Service]
 LIB --> PY[Python services\nnai_service / insights_service]
 PY --> EXT[External fact-check/search providers]

U --> FE FE --> API FE --> LIB API --> LIB API --> DB LIB --> SM LIB --> OLLAMA LIB --> DEEP LIB --> NEWS LIB --> RAG LIB --> PY LIB --> EXT PY --> OLLAMA PY --> DB

Frontend layer

The frontend is built with:

- Next.js 16
- React 19
- App Router pages under `src/app`
- Tailwind-style CSS via `src/app/globals.css` and component classes
- Session context via `next-auth/react` provider in `src/app/providers.tsx`

The app shell is defined in `src/app/layout.tsx` and includes:

- Navbar
- Footer
- Session provider
- main content wrapper

Backend layer

The main backend is Next.js route handlers under `src/app/api`. Server files call Prisma for persistence

Examples of the main backend responsibilities:

- blog read/write flow
- auth and session handling
- commentary lifecycle
- scoring pipeline
- leaderboard and writer profile updates
- contest moderation
- live and historical match analysis

Shared logic layer

The src/lib folder contains most reusable logic:

- auth.ts – NextAuth and credentials/Google provider setup
- db.ts – Prisma client singleton
- cricket-api.ts – SportMonks or legacy fallback fetches
- scoring.ts – BQS/quality pipeline and writer DNA logic
- fact-check.ts – claim extraction and fact checking
- historical-warehouse.ts – structured historical match analysis
- internal-originality.ts – internal duplicate detection
- deepgram.ts – speech-to-text integration
- commentary-* files – commentary tempo and formatting

Python services layer

There are dedicated Python directories:

- ai_service/
- insights_service/

These are not just helper scripts; they provide service endpoints for AI tasks and analysis flows. Th

Data flow by domain

Match content flow

- User navigates to a match page or home page
- Frontend calls match APIs
- src/lib/cricket-api.ts resolves data source (SportMonks first, fallback to legacy API, then mock mo
- Results are normalized into the app's Match/Scorecard types
- UI displays scorecards, commentary, squads, and analysis

Blog publishing flow

- User writes a blog in src/app/blog/write/page.tsx
- Client triggers AI tag generation and optional polishing
- Route POST /api/blogs creates a post
- Embedding generation runs for internal originality checks
- Scoring pipeline computes BQS and related metrics
- Blog score and writer profile are updated in Prisma

Commentary flow

- User starts or joins a commentary session
- Session is recorded in Prisma
- Audio is transcribed through Deepgram or AI service proxy
- Player names are normalized and commentary text is polished
- Commentary entries are stored and served by API routes

Implementation status notes

The architecture is coherent and modular, but there are several configuration-sensitive dependencies:

- live match data requires SportMonks token or fallback API keys
- AI analysis depends on Ollama being reachable and configured
- Python AI services must be available in production or local dev environments
- database provider setup is not fully consistent across files

Architectural strengths

- Clear separation between pages, routes, and libs
- Strong service encapsulation around external APIs
- Prisma model layer gives a consistent persistence boundary
- AI workflows are modularized and partially isolated into separate services

Architectural risks

- Environment-sensitive runtime dependencies make local setup brittle
- SQL Server vs MySQL configuration mismatch in schema/example env files
- Some features are tied to demo fallback logic rather than full production flows
- Large amount of AI logic is integrated across Node and Python code with implicit env assumptions

PROJECT STRUCTURE

02-PROJECT-STRUCTURE

Actual repository structure

The workspace contains a monorepo-like combination of a Next.js app, Python AI services, operational docs, scripts, and Prisma definitions.

```
cricgeek-app/  
  .env, .env.example, .env.production.example  
  .github/  
  .next/  
  ai_service/  
  analysis/  
  capstone cric/  
  cloudflare/  
  data/  
  docs/  
  frontend/  
  insights_service/  
  launchd/  
  ollama-proxy/  
  ops-backups/  
  prisma/  
  public/  
  rag/  
  reports/  
  scripts/  
  src/  
  eslint.config.mjs  
  next-env.d.ts  
  next.config.ts  
  package.json  
  package-lock.json  
  postcss.config.mjs  
  README.md  
  requirements.txt  
  tsconfig.json  
  vercel.json  
  ...
```

Top-level functional areas

src/

This is the main application codebase.

Relevant folders:

- src/app — App Router pages, layouts, and route handlers
- src/components — reusable UI components
- src/lib — application logic, API integration, scoring, auth, data access
- src/types — typed cricket data models and interfaces
- src/hooks — client-side hooks (if present)

prisma/

Contains the Prisma schema and SQL migration scripts.

Notable files:

- prisma/schema.prisma — source of truth for database model types
- prisma/create_tables.sql — SQL creation script
- prisma/comment_threads_migration.sql
- prisma/contest_engine_migration.sql
- prisma/fact_check_search_migration.sql
- prisma/historical_warehouse_migration.sql

ai_service/

Python services used as the AI backend for analysis, fact checking, and model-backed features. Examples:

- ai_service/claim_extractor.py
- ai_service/fact_checker.py
- ai_service/ollama_client.py
- ai_service/ranking_service.py
- ai_service/research_agent.py
- ai_service/scoring.py
- ai_service/verification_agents.py

insights_service/

Python service related to match insights and analytics processing. Example files:

- insights_service/main.py
- insights_service/t20_api.py
- insights_service/t20_insights.py
- insights_service/cricket_metrics.py
- insights_service/evaluator.py

rag/

Contains retrieval-augmented generation support for contextual search.

scripts/

Operational scripts, import pipelines, and debugging flows.

docs/

Contains project documentation, deployment guides, and setup notes.

Source map by responsibility

Frontend pages

Under src/app:

- landing page: src/app/page.tsx
- blog pages: src/app/blog, src/app/blog/[slug]

- match pages: `src/app/matches`, `src/app/matches/[id]`
- commentary pages: `src/app/commentary`, `src/app/commentary/[sessionId]`
- writer pages: `src/app/writer/[id]`
- auth pages: `src/app/auth/login`, `src/app/auth/register`
- admin page: `src/app/admin/page.tsx`
- legal pages and content policy pages

API layer

Under `src/app/api`:

- auth routes
- blog routes
- commentary routes
- matches routes
- insights routes
- admin routes
- AI routes for tags, EQS, paraphrase, upload
- scoring validation/analyze routes

Shared domain logic

Files under `src/lib`:

- `cricket-api.ts` — match data provider abstraction
- `scoring.ts` — overall quality scoring engine
- `fact-check.ts` — fact checking flow
- `historical-warehouse.ts` — structured historical data analysis
- `deepgram.ts` — transcription support
- `commentary-polish.ts` — commentary text cleanup
- `auth.ts` — NextAuth config and user orchestration
- `contest.ts` — contest leaderboard logic

Relationship between app layers

Application flows usually map like this:

- Page component in `src/app`
- API route under `src/app/api`
- Domain helper in `src/lib`
- Prisma query or persistence operation
- external provider call
- response returned to client

Important implementation observations

Layering is coherent but broad

The app uses a single large lib folder rather than strict domain submodules. The following directory patterns exist, but the code is not split into a strict layered architecture with separate controllers and repos.

AI modules are split across Node and Python

The architecture is hybrid, which means developers must understand both the Node runtime and Python service environment. This is a key operational requirement.

Some features are still partially demo-driven

The demo-data.ts file and fallback paths in auth and blog flows show a strong local/demo experience, which indicates the app is intentionally resilient during startup or when dependencies are missing.

FRONTEND

03-FRONTEND

Frontend stack

The primary frontend is built with:

- Next.js 16
- React 19
- TypeScript
- Tailwind-style utility classes
- next-auth/react for session handling

The app shell is created in `src/app/layout.tsx`.

App router structure

The main pages under `src/app` include:

- `/` — home landing page
- `/matches` — match listings and live hub
- `/matches/[id]` — match detail view
- `/blog` — expression feed
- `/blog/[slug]` — article detail page
- `/blog/write` — drafting experience and EQS scoring flow
- `/commentary` — live commentary sessions
- `/leaderboard` — writer ranking table
- `/writer/[id]` — writer profile page
- `/admin` — admin dashboard
- `/auth/login` and `/auth/register` — authentication screens
- `/contact` and legal pages

Page behavior overview

Home page

File: `src/app/page.tsx`

Purpose:

- expose landing page hero and feature summary
- show live matches, recent results, and upcoming fixtures
- link to blogs, insights, and calendar flows

Data source:

- `getMatchHubMatches()` in `src/lib/cricket-api.ts`

Matches page

File: `src/app/matches/page.tsx`

Purpose:

- list the current match hub
- pass data to `MatchesClient`

Match detail pages

Files:

- `src/app/matches/[id]/page.tsx`
- `src/app/matches/[id]/preview/page.tsx`
- `src/app/matches/[id]/analysis/page.tsx`
- `src/app/matches/[id]/post-match/page.tsx`

Purpose:

- show scorecards, previews, analysis, and post-match detail
- route to tabs such as scorecard, commentary, squads, analysis

Blog feed and post pages

Files:

- `src/app/blog/page.tsx`
- `src/app/blog/[slug]/page.tsx`
- `src/app/blog/write/page.tsx`

Purpose:

- allow user-authored cricket expressions
- allow social reactions, saves, comments, and leaderboard-linked visibility
- includes a draft editor with EQS scoring flow

Commentary pages

Files:

- `src/app/commentary/page.tsx`
- `src/app/commentary/[sessionId]/page.tsx`

Purpose:

- list commentary sessions and existing session state
- support moderator session management and display of commentary entries

Writer pages

Files:

- `src/app/writer/[id]/page.tsx`
- `src/app/leaderboard/route.ts` (API) and leaderboard page if present under `src/app/leaderboard` maybe no page file in tree but likely route exists

Purpose:

- display writer profile, achievements, badging, DNA, and recent blogs

Major reusable component groups

Layout

- `src/components/layout/Navbar.tsx`
- `src/components/layout/Footer.tsx`

Matches

- `LiveMatchCard`
- `MatchLiveCommentary`
- `LiveScoresTicker`
- `MatchDetailClient` and related UI blocks

Blog

- `CricketBallReactionButton`
- `SaveBlogButton`
- `BlogDiscussion`
- score / breakdown display UI

Writer

- `WriterProfileCard`
- `WriterDNAChart`
- `FollowWriterButton`
- `ScoreRing`

Ads

- `src/components/ads/AdSlot.tsx`

Frontend state patterns

The app uses a mostly server-first pattern with client-side state in `pages/components` for:

- blog search and feed selection
- auth session state
- live commentary sessions
- EQS publishing progress
- draft persistence to `localStorage`
- ad slot / UI toggles

The app provider wraps the app in `SessionProvider` from `next-auth/react` in `src/app/providers.tsx`.

Data access pattern

Client components usually fetch through route handlers in `src/app/api` and then render normalized results in the browser.

Examples:

- `fetch('/api/blogs')` to load expressions
- `fetch('/api/matches')` or direct page server fetches for live match hub
- `fetch('/api/commentary')` to fetch commentary session list
- `fetch('/api/writer/[id]')` for writer profile data

UI behavior and validation patterns

Several components implement client-side checks for:

- word count limits for blog posting
- status toggles and loading states
- empty-state rendering
- auth gating for session-based flows
- contest and admin UI states

Frontend status notes

The frontend is clearly active and feature-rich, but several user flows depend on external services or backend env configuration. The strongest example is the writing flow, which requires blog creation, scoring pipeline, and AI inference to be configured.

Notable frontend risks

- A large number of UI flows assume backend services are available without strong graceful degradation
- Some pages appear designed for local demo user flows and may not behave cleanly without seeded DB data
- The admin UI is role-based but depends on actual session role values being present in the DB and auth system

BACKEND

04-BACKEND

Backend runtime

The primary backend is the Next.js App Router runtime in `src/app/api` and the server-side modules in `src/lib`. The app also includes Python services under `ai_service` and `insights_service`.

Framework and server stack

- Next.js 16 App Router
- React 19
- Prisma ORM
- NextAuth
- optional Python microservices for AI and insights

Request lifecycle

Typical request lifecycle for a route:

1. Browser requests a page or API route 2. Route handler in `src/app/api` executes 3. Route uses `auth()` or session checks when needed 4. Route reads/writes Prisma models 5. Route calls helper logic in `src/lib` or third-party APIs 6. JSON or redirect response is returned to the client

Main backend responsibilities

Authentication and identity

- `src/lib/auth.ts`
- `src/app/api/auth/login/route.ts`
- `src/app/api/auth/register/route.ts`
- `src/app/api/auth/[...nextauth]/route.ts`

This provides:

- credentials auth with `bcrypt`
- optional Google auth provider
- demo fallback via `src/lib/demo-data.ts`
- role-aware session data

Blogs and content lifecycle

- `src/app/api/blogs/route.ts`
- `src/app/api/blogs/[slug]/route.ts`
- `src/app/api/blogs/[slug]/comments/route.ts`
- `src/app/api/blogs/[slug]/comments/route.ts`

- `src/app/api/blogs/[slug]/save/route.ts`
- `src/app/api/blogs/[slug]/runs/route.ts`
- `src/app/api/blogs/[slug]/views/route.ts`
- `src/app/api/blogs/report/route.ts`

This covers:

- blog creation and listing
- comment handling
- view tracking
- reactions and saves
- report submission

Match and cricket data

- `src/app/api/matches/route.ts`
- `src/app/api/matches/[id]/live/route.ts`
- `src/app/api/livescores/route.ts`
- `src/app/api/insights/live/route.ts`
- `src/lib/cricket-api.ts`
- `src/lib/sportmonks.ts`

This layer normalizes match data from SportMonks or fallback sources and feeds UI pages.

Commentary

- `src/app/api/commentary/route.ts`
- `src/app/api/commentary/[sessionId]/route.ts`
- `src/app/api/commentary/[sessionId]/entries/route.ts`
- `src/app/api/commentary/[sessionId]/stream/route.ts`
- `src/app/api/commentary/transcribe/route.ts`

This covers:

- session creation and lifecycle
- live entries
- audio transcription via Deepgram or AI service
- cleanup and formatting of commentary text

AI and scoring

- `src/app/api/scoring/analyze/route.ts`
- `src/app/api/scoring/validate/route.ts`
- `src/app/api/ai/eqs/route.ts`
- `src/app/api/ai/tags/route.ts`
- `src/app/api/ai/paraphrase/route.ts`
- `src/app/api/ai/upload/route.ts`

- `src/lib/scoring.ts`
- `src/lib/local-ai.ts`
- `src/lib/fact-check.ts`
- `src/lib/internal-originality.ts`

These routes produce and update the quality score pipeline and EQS outputs.

Admin and moderation

- `src/app/api/admin/blogs/route.ts`
- `src/app/api/admin/contests/route.ts`
- `src/app/api/admin/seed-historical-warehouse/route.ts`

These routes support moderation and contest administration.

Service layer design

The backend is not separated into strict controllers/repositories; rather, logic is concentrated into route files and lib modules.

This means the code base depends on domain helper functions for:

- `auth`
- `scoring`
- `fact checking`
- `data provider access`
- `historical warehouse operations`
- `commentary transformation`

Error handling patterns

Common patterns seen across the backend:

- `parse/validate request body` and return 400 on bad input
- `auth checks` return 401 or 403
- `DB or API failure` uses 500 and logs errors
- `service failure` may degrade to demo fallback data or mocked values

External backend dependencies

The app expects external services for many capabilities:

- `SportMonks API` for cricket data
- `Ollama` for LLM-based model analysis
- `Deepgram` for STT
- `GNews / TheNewsAPI` for news-related features
- `Tavily or Serper` for fact checking
- `Python AI service host` for legacy features
- `RAG service` for retrieval features

Backend status notes

The backend is functionally rich and has a coherent general design, but the system is not fully self-contained. Several critical flows require configured external infrastructure to work beyond demo or fallback behavior.

Notable backend concerns

- environment config is essential for production operation
- hybrid Node/Python runtime introduces operational complexity
- some fallback logic reads as progressive enhancement rather than production-safe failover
- DB mismatch between SQL Server schema and MySQL examples indicates ongoing configuration drift

DATABASE

05-DATABASE

Database layer

The data layer is driven by Prisma and defined in `prisma/schema.prisma`.

The schema contains a SQL Server datasource, which is the actual configured provider for the project.

```
datasource db {
  provider = "sqlserver"
  url      = env("DATABASE_URL")
}
```

This means the main app is not currently configured as SQLite-based. Some helper docs or examples may mention other databases, but the schema is the authoritative source.

Core entities

User

Represents a platform user, account identity, and writer profile.

Primary fields likely include:

- id
- name
- email
- passwordHash
- image
- role
- createdAt
- updatedAt

Related models:

- WriterProfile
- Blog
- CommentThread
- Comment
- ContestSubmission
- Follow
- Save

Blog

A single cricket expression/article.

Important fields include:

- id
- title
- slug
- content
- excerpt
- status
- authorId
- createdAt
- updatedAt

Relationships:

- author -> User
- score -> BlogScore
- comments -> comments and threads
- reports -> report records
- saves
- reactions
- contest submissions

BlogScore

Stores the computed evaluation and score snapshot for a blog.

This is a critical model for the EQS pipeline and includes fields like:

- bqs
- toneScore
- negativityScore
- toxicityScore
- originalityScore
- coherenceScore
- constructiveness
- evidencePresence
- infoDensity
- argumentLogic
- factCheckOverallScore
- processingStatus
- scoreVersion

It is the canonical place where article quality metrics are persisted.

WriterProfile

Stores aggregated writer leaderboards and reputation metrics.

Important fields include:

- averageBQS
- totalBlogs
- totalViews
- totalRuns
- archetype
- writerTitle
- level
- xp
- bestBQS
- featuredCount
- streak
- bcs
- statAccuracy

This model supports leaderboard and achievement systems.

WriterDNA

Tracks a writer's stylistic profile or archetype composition.

Likely used to compute how a writer tends to write:

- analyst
- storyteller
- fan
- debater

HistoricalMatch and other analytics models

The schema includes models for historical cricket data and warehouse analysis.

These likely support:

- historical fact-checking
- match analysis
- statistical normalization
- replay-style or post-match review flows

Contest and contest submissions

Contest features are represented by models like:

- Contest
- ContestSubmission

These track:

- contest title and dates
- rules, prize, deadlines

- submission blog relationships
- ranking, scores, final score, overrides
- announcements and status

Commentary and sessions

The app includes a commentary domain important for live match discussion and voice commentary. Models such as:

- LiveCommentarySession
- CommentaryEntry

are present in the schema and support session-based live commentary streams.

Relationship patterns

The schema is designed around a social publishing model:

- user -> blogs
- user -> writerProfile
- blog -> blogScore
- blog -> comments / reactions / saves / reports
- user -> follows
- contest -> submissions

This gives the platform both product interaction and gamification features in one database.

Database risks and inconsistency

There are several important configuration concerns:

- schema uses SQL Server, but README and environment examples may refer to SQLite-like or generic local setups
- some SQL migration files may show older or alternative schema assumptions
- production readiness depends on migration synchronization between schema.prisma and actual deployed DB

Operational guidance

For a new developer, the primary DB tasks are:

1. Ensure DATABASE_URL is correctly set for the selected environment
2. Run Prisma migration or schema sync against the target database
3. Validate the generated Prisma client after schema changes
4. Check connection string compatibility with SQL Server requirements

Database status

The database layer is real and structurally rich. It is one of the core strengths of the application, although operational drift between docs and actual provider config should be treated as a deployment risk.

AUTHENTICATION SECURITY

06-AUTHENTICATION-SECURITY

Current auth model

Authentication is implemented through NextAuth and the auth configuration in `src/lib/auth.ts`.

The app supports:

- credentials login
- Google OAuth provider
- session-based auth with role metadata
- admin gating patterns on route handlers and pages

Provider setup

The auth flow uses:

- NextAuth.js configuration
- CredentialsProvider
- GoogleProvider
- custom user/role mapping logic

The code includes user normalization and fallback logic for demo or local-development flows.

Role model

The app includes a role concept used for admin access.

Example admin gating pattern:

```
const session = await auth();
const user = session?.user as { role?: string } | undefined;
return user?.role === "admin";
```

This role check is used in admin route handlers such as:

- `src/app/api/admin/blogs/route.ts`
- `src/app/api/admin/contests/route.ts`

Security posture

The code includes common protections such as:

- server-side auth checks for administrative actions
- input validation before DB updates
- no direct client-side permission override in the API layer

However, this repo does not yet show a strong enterprise-grade security architecture document with:

- RBAC matrix
- fine-grained permission checks for all routes

- centralized authorization middleware
- audit log retention for moderation actions

Demo and local development behavior

There are signs that the app is designed for demo readiness or quick local setup:

- default or fallback credentials in some auth flows
- mock user data in demo scenarios
- UI fallback states when a user is not signed in

This is helpful for development, but it means production behavior must be reviewed before deployment to avoid permissive dev assumptions.

Env-sensitive security settings

The app depends on several environment variables, including:

- NEXTAUTH_SECRET
- NEXTAUTH_URL
- DATABASE_URL
- Google credentials if used
- service-specific tokens for external APIs

Without these values, auth and supporting features may fail or degrade.

Risk areas

- role values must be trusted and populated correctly in the app database
- production secret rotation has to be handled externally
- admin-only routes depend on server-side session values, so DB state and auth config must stay in sync
- OAuth or credentials configuration drift can create silent auth failures

Authentication status

Authentication is implemented and functional at a basic app level, but the production security posture should be reviewed against a stricter RBAC and deployment checklist before release.

API REFERENCE

07-API-REFERENCE

API style

This application uses Next.js route handlers for its backend API surface. Most routes return JSON and often rely on Prisma and auth gating.

Route group categories

Authentication routes

- /api/auth/[...nextauth]
- /api/auth/login
- /api/auth/register

Purpose:

- session establishment
- credentials / OAuth login
- account creation and user onboarding

Blog routes

- GET /api/blogs
- POST /api/blogs
- GET /api/blogs/[slug]
- PATCH /api/blogs/[slug]
- POST /api/blogs/[slug]/comments
- POST /api/blogs/[slug]/save
- POST /api/blogs/[slug]/views
- POST /api/blogs/[slug]/runs
- POST /api/blogs/report

Purpose:

- blog discovery, creation, publication, and moderation
- comment threads, saves, reactions, views, and reporting

Match routes

- GET /api/matches
- GET /api/matches/[id]
- GET /api/matches/[id]/live
- GET /api/matches/[id]/scorecard
- GET /api/insights/live

Purpose:

- fetch live and historical match data
- return match detail bundles
- provide scorecard and commentary context

Commentary routes

- GET /api/commentary
- POST /api/commentary
- GET /api/commentary/[sessionId]
- POST /api/commentary/[sessionId]/entries
- POST /api/commentary/transcribe

Purpose:

- commentary session creation and participation
- transcription and polish flows

AI routes

- POST /api/ai/eqs
- POST /api/ai/tags
- POST /api/ai/paraphrase
- POST /api/ai/upload
- POST /api/scoring/analyze
- POST /api/scoring/validate

Purpose:

- analyze and score blog quality
- generate tags and text rewrites
- integrate with local AI or service-based inference

Admin routes

- GET /api/admin/blogs
- PATCH /api/admin/blogs
- GET /api/admin/contests
- POST /api/admin/contests
- PATCH /api/admin/contests

Purpose:

- moderate blog submissions
- configure contests
- recompute standings
- publish announcements

Common response patterns

Typical responses are either:

- JSON object with a result payload
- status code 200/201/400/403/500
- error message objects on failure

Example response structure:

```
{
  "message": "Blog updated",
  "blog": { "id": "..." }
}
```

Authenticated endpoints

Many admin, blog write, and contest-related APIs require session-bound auth. The app commonly uses `auth()` to verify role and identity before proceeding.

Notes on API robustness

The API layer is active but not uniformly fully hardened.

Main concerns:

- validation is inconsistent across routes
- some routes are environment dependent
- some endpoints have fallbacks for demo mode and local testing
- some data fetches assume upstream service health

Example backend logic

The following route pattern is common:

```
export async function POST(req: NextRequest) {
  try {
    const body = await req.json();
    // validate
    // call Prisma or external service
    // return JSON
  } catch (error) {
    console.error("error", error);
    return NextResponse.json({ error: "Failed" }, { status: 500 });
  }
}
```

This indicates a straightforward server-route design rather than GraphQL or an explicit REST specification.

API status

The API surface is substantial and coherent, but it is better understood as a product API with partial service integration than as a fully standardized public API contract.

PAGES

08-PAGES

Major app routes

This section summarizes the primary page responsibilities in the app.

/ (home)

Acts as a landing page and hub for match updates, editorial promos, and community features.

/matches

Lists current and recent matches, often sourced from cricket data provider wrappers.

/matches/[id]

Shows the detailed match experience, including tabs for scorecard, commentary, squads, and analysis.

/blog

Displays the community feed for cricket expressions and blog articles.

/blog/[slug]

Displays a single blog article with score breakdown, author profile, reactions, comments, and fact-check visualization.

/blog/write

Primary writing workflow page. Supports:

- draft composition
- AI editing and tag generation
- quality scoring
- publishing and tracking

/commentary

Provides a commentary experience and list of available sessions.

/commentary/[sessionId]

The feature page for a single live commentary session.

/leaderboard

Displays writer rankings and top blog/performance information.

/writer/[id]

Writer identity and profile page showing:

- DNA

- badges
- achievements
- recent expressions
- score metrics

/admin

Admin dashboard for moderation and contest management.

/auth/login

Login screen and auth UI.

/auth/register

Registration flow for new accounts.

Additional route groups

These pages include legal, marketing, and support content:

- /contact
- /legal/terms
- /legal/privacy
- /help and support pages

Page implementation notes

Several pages are implemented as client components when they need interactive experiences for:

- scoring status
- editing drafts
- live dashboards
- social actions

This means page behavior is often distributed between server components and client components depending on interactivity needs.

Page-level dependencies

Key dependencies include:

- Prisma data access
- NextAuth session checks
- SportMonks or legacy match wrappers
- scoring service and AI endpoints
- Python insights service endpoints

Status summary

The page map is broad and feature-rich. The most central user journeys are the writing, match, profile, and admin experiences—these are the highest-signal parts of the product.

FEATURES

09-FEATURES

Product feature map

The app combines sports content, social writing, community gamification, and AI analysis. The main product domains are:

- live cricket match information
- blog publishing and community scoring
- writer progression and leaderboard systems
- commentary and transcription flows
- contests and moderation
- fact-checking and historical analysis

1. Match intelligence

CricGeek includes live data and match-specific UIs for:

- list of live matches
- scorecards
- commentary logs
- squad information
- match-specific analysis pages

This is driven primarily through SportMonks integration with fallback logic in `src/lib/cricket-api.ts` and `src/lib/sportmonks.ts`.

2. Writing and publishing

The blog and writing workflow is one of the app's core product features. Users can:

- draft content
- generate tags and AI polishing
- evaluate with EQS
- publish to community feed
- receive scoring and profile impact

Implementation core:

- `src/app/blog/write/page.tsx`
- `src/lib/scoring.ts`
- `src/lib/local-ai.ts`
- `src/app/api/blogs/route.ts`

3. EQS and quality scoring

The app has a significant scoring engine called the EQS or evidence-quality style pipeline. It computes:

- BQS (Blog Quality Score)
- tone and negativity metrics
- toxicity and originality evaluation
- evidence presence and argument logic
- archetype classification
- writer performance impact

This is implemented in `src/lib/scoring.ts` and route handlers under `src/app/api/ai` and `src/app/api/scoring`.

4. Writer identity and community

The app includes a rich writer community layer:

- profile pages
- badges and achievements
- follow/follower relationships
- writer DNA visualization
- statistics and trend metrics

Core files:

- `src/app/writer/[id]/page.tsx`
- `src/components/writer/WriterProfileCard.tsx`
- `src/lib/scoring.ts`

5. Contest engine

A contest module exists for challenge-based writing competition.

Features include:

- contest creation by admins
- submission scoring
- leaderboard recomputation
- override scores
- announcement publishing

Core files:

- `src/lib/contest.ts`
- `src/app/api/admin/contests/route.ts`
- `src/components/admin` dashboard code in `src/app/admin/page.tsx`

6. Fact checking and claim validation

The app supports claim extraction and fact validation with routing between:

- historical structured lookup
- web search
- unsupported claim handling

Core files:

- `src/lib/fact-check.ts`
- `ai_service/fact_checker.py`
- `ai_service/claim_extractor.py`

7. Voice to commentary

The app includes a voice and transcription workflow for commentary generation or processing.

Feature focus:

- audio submission
- speech-to-text conversion
- commentary polishing and cleaning
- session storage for live commentary

Core files:

- `src/app/api/commentary/transcribe/route.ts`
- `src/lib/deepgram.ts`
- `src/app/commentary` pages and components

8. Historical and analytics layers

Historical warehouse models and analysis services appear to support:

- claim recall against historical match data
- structured fact validation
- deeper match analysis and replay-like evaluation

This is visible through Prisma models and scripts under historical warehouse logic.

9. Operational/admin tooling

The app also includes a moderation and admin layer with:

- blog status updates
- contest management
- featured content review
- scoring pipeline visibility

Product maturity assessment

The app is not a narrow single-feature prototype. It is a multi-feature sports-content platform with a layered and fairly rich domain model.

However, its feature set assumes a combination of:

- configured database
- configured auth
- AI model availability
- cricket data provider access

- operational service hosting

Without those, the app falls back to demo patterns or partial functionality.

EQS

10-EQS

Overview

EQS refers to the app's quality-scoring framework for blog content. It is one of the most central features in the platform and drives writer performance, trust signals, and reward logic.

This scoring system lives mainly in:

- `src/lib/scoring.ts`
- `src/lib/local-ai.ts`
- `src/lib/internal-originality.ts`
- `src/app/api/ai/eqs/route.ts`
- `src/app/api/scoring/analyze/route.ts`

What EQS evaluates

The scoring model evaluates content against a broad set of dimensions:

- `toneScore`
- `negativityScore`
- `toxicityScore`
- `originalityScore`
- `coherenceScore`
- `constructiveness`
- `evidencePresence`
- `positionClarity`
- `infoDensity`
- `argumentLogic`
- `statsFound`
- `statsVerified`
- `statAccuracy`
- `BQS`

The app then maps these into a final score and often a writer archetype.

Deterministic and AI-backed scoring

The scoring pipeline is hybrid:

- heuristic analysis of structure, tone, negativity, and evidence density
- AI model-based analysis via Ollama or AI service
- originality check using internal duplicate detection

- fact-check scoring to penalize unsupported claims

This hybrid pattern is essential because a single model is not the only determinant of quality.

Internal originality module

The internal originality checker uses a token/embedding-like process to detect repeated content patterns inside the app. It is designed to prevent low-originality writing from being rewarded.

Core logic lives in `src/lib/internal-originality.ts` and includes:

- tokenization
- shingling or hash-like embedding generation
- cosine similarity comparisons
- originality scoring output

This supports the system's anti-duplicate and anti-rehash quality controls.

Local AI model integration

The local AI layer in `src/lib/local-ai.ts` performs a model-backed scoring pass with Ollama. This is the main path for tool-assisted overall quality evaluation.

Notable characteristics:

- JSON output parsing from model responses
- heuristic fallback when model output is parseable but invalid or unavailable
- multiple checks before returning a score

Route-level implementation

The app exposes a dedicated endpoint:

- `src/app/api/ai/eqs/route.ts`

This route explicitly declares Node runtime:

```
export const runtime = "nodejs";
```

This confirms the main EQS route is not configured as an Edge runtime endpoint.

Writer impact

Once a blog is scored, the system updates:

- `blogScore` records
- `writerProfile` metrics
- achievements / badge triggers
- total votes, views, and engagement metrics

This creates a strong loop where publishing quality directly affects writer status.

EQS maturity

The EQS system is one of the most developed parts of the codebase and is more than a basic heuristic engine. It clearly intends to be a central editorial quality arbitration tool.

However, to be fully reliable in production it still depends on:

- model availability
- consistent data quality
- robust fallback behavior
- external fact-check service health

EQS status

IMPLEMENTED and central to the platform, but operationally dependent on configured AI and external verification services.

VOICE TO COMMENTARY

11-VOICE-TO-COMMENTARY

Overview

The app contains a voice-to-commentary or audio commentary flow, designed to convert spoken cricket commentary into stored and polished text.

This functionality is split across:

- `src/app/api/commentary/transcribe/route.ts`
- `src/lib/deepgram.ts`
- commentary session routes
- related UI under `src/app/commentary` and component folders

Supported flow

The intended flow is:

1. User submits or records commentary audio 2. Audio is sent to a transcription provider 3. The raw transcript is normalized and polished for cricket-specific wording 4. The resulting commentary text is stored to a session or blog-like commentary record 5. The UI shows the commentary stream

Transcription providers

The code includes Deepgram support through `src/lib/deepgram.ts`. There are also fallback patterns for AI-based transcription or polishing when external services are unavailable.

This indicates the app expects a layered transcription strategy rather than a single hard dependency.

Commentary session model

The Prisma schema includes live commentary-specific tables and session state, allowing the app to model:

- commentary sessions
- session memberships or participants
- commentary entries
- voice and text events

Operational dependencies

This workflow depends on:

- Deepgram API key or valid fallback service
- valid audio uploads or capture paths
- optional AI polishing logic
- database persistence for commentary sessions

Status

The feature is implemented in a substantial way, but the real runtime depends on configured external transcription services and careful audio flow management.

This is a functional product area, but not one that can be assumed to work in a local environment without the required service credentials.

EXTERNAL SERVICES

12-EXTERNAL-SERVICES

External service inventory

The app is explicitly dependent on several external providers and runtime services.

Cricket data provider

- SportMonks
- Used through `src/lib/cricket-api.ts` and `src/lib/sportmonks.ts`
- Provides live fixtures, scorecards, and match metadata

Model inference provider

- Ollama
- Used for local AI scoring and text analysis paths
- Qwen-based scoring is referenced in docs and code comments

Speech transcription

- Deepgram
- Used for audio-to-text commentary transcription
- Configured via `src/lib/deepgram.ts`

Search and fact checking

- Tavily
- Serper
- optional GNews or news search integration
- Used in `src/lib/fact-check.ts` for claims and verdict synthesis

Python services

- `ai_service`
- `insights_service`
- local FastAPI or Python services are expected to respond at configured service URLs

Optional cloud or deployment integrations

- Cloudflare, Vercel, and tunnel configuration docs exist
- This indicates production deployment and reverse-proxy concerns are part of the platform's broader ops picture

Service configuration pattern

The app typically expects environment-based configuration like:

- `DATABASE_URL`

- SPORTMONKS_API_TOKEN
- OLLAMA_URL
- AI_SERVICE_URL
- INSIGHTS_URL / T20_INSIGHTS_URL
- DEEPGRAM_API_KEY
- GNEWS_API_KEY
- NEXTAUTH_SECRET

This means production readiness is strongly coupled to external service health and env correctness.

Operational risk

Service outages or misconfiguration do not just affect one feature—they can degrade the app's core user experience because many flows are built around remote providers.

Service status

The external integration layer is an important part of the system. It is functional in a configured environment, but it is not self-contained and requires infrastructure planning before deployment.

CONFIGURATION

13-CONFIGURATION

Environment configuration overview

The project depends on environment variables for database access, auth, external APIs, and AI service connectivity.

The codebase includes files such as:

- .env.example
- .env.production.example
- README.md
- docs/ setup guides

Core env variables

The app expects values such as:

- DATABASE_URL
- NEXTAUTH_SECRET
- NEXTAUTH_URL
- GOOGLE_CLIENT_ID
- GOOGLE_CLIENT_SECRET
- SPORTMONKS_API_TOKEN
- OLLAMA_URL
- AI_SERVICE_URL
- INSIGHTS_URL or T20_INSIGHTS_URL
- DEEPGRAM_API_KEY
- GNEWS_API_KEY
- TAVILY_API_KEY / SERPER_API_KEY if used

Runtime configuration notes

Database

The main schema is for SQL Server, so the database config should be aligned with the current deployed DB.

Auth

Auth settings are required for both local dev and production. If they are absent, sign-in and admin flows will fail or degrade.

AI and facts

AI scoring and fact-check flows are often gated by service health and env values. Without a working model or service backend, those features either fall back to heuristics or become unavailable.

Local development considerations

The repo supports local dev but expects environment files and possibly background services such as:

- Ollama
- Python AI service
- database instance
- match API connectivity

Deployment configuration notes

The repo includes deployment-related docs and config such as:

- vercel.json
- cloudflare/
- launchd/
- docs for tunnel and production setup

This signals that operations and deployment setup are treated as first-class concerns for the project.

Configuration risk

The main config issue is not a single missing variable—it is the broad dependency chain across database, auth, AI, and external provider configuration. This makes startup and deployment sensitive to environment correctness.

DEPENDENCIES

14-DEPENDENCIES

Node / Next.js runtime

The app is built on:

- Next.js 16
- React 19
- TypeScript
- Tailwind CSS
- Prisma
- NextAuth

Key packages in the package manifest include:

- next
- react
- react-dom
- @prisma/client
- prisma
- next-auth
- lucide-react
- zod or validation libraries
- Playwright or testing components
- other app dependencies added during feature development

Python service dependency notes

The Python space includes additional libraries, mostly under the separate requirements files:

- requirements.txt at repo root
- ai_service/requirements.txt
- insights_service/requirements.txt

These are needed for:

- AI model interaction
- data processing
- analytics
- fact verification
- retrieval and inference flows

External runtime dependencies

The product depends on runtime infrastructure outside the Node project:

- SQL Server or compatible DB
- Ollama service
- optional FastAPI service endpoints
- SportMonks API token and data access
- Deepgram token
- search provider tokens

Dependency status

The dependency model is broad and service-heavy, which is common for a product with AI and real-time sports data integration. The main challenge is not missing packages but missing runtime environment readiness.

IMPLEMENTATION STATUS

15-IMPLEMENTATION-STATUS

Summary of implementation status

This repo contains a substantial product implementation with many connected flows.

Implemented and active

- Next.js app shell and UI
- Prisma schema and DB models
- Blog publishing and feed flows
- Writer profile and leaderboard logic
- Scoring and EQS engine
- Contest management
- Admin tooling
- Fact-check and claim validation flows
- Match data integration
- Voice/transcription commentary support

Partially implemented or environment dependent

- Full AI scoring and evaluation requires external model connectivity
- Production-grade auth setup depends on correct env and session configuration
- Historical warehouse and advanced structured facts require service/data health
- Live commentary flows depend on voice/transcription infrastructure

Planned or not clearly wired

Some remarks in the repo suggest ideas for future work, including:

- extended replay grading
- more advanced historical analytics
- additional operational or deployment improvements

These are not central to the active app flow and should be treated as future extension work unless explicitly wired into current code.

Confidence level

This is a real, active product codebase with a clear architecture, but it should be treated as a configured-service application rather than a purely static frontend or toy app.

Recommended onboarding approach

1. Start with project config and env files 2. Validate database connectivity 3. Confirm auth and admin role setup 4. Start Ollama or the AI service 5. Confirm SportMonks and search provider config 6. Run the app locally and validate key user journeys

Final status assessment

The application's core implementation is substantial and coherent. The most significant operational requirement is ensuring the required service environment is available.

KNOWN ISSUES

16-KNOWN-ISSUES

Confirmed issues and operational risks

1. Runtime confusion: Edge vs Node

The app does not implement an Edge runtime at the EQS route level. The route uses:

```
export const runtime = "nodejs";
```

This means the system is built around the standard Node.js server runtime, not Edge runtime, unless a route or deployment specifically overrides it elsewhere.

2. Environment-sensitive startup

Many features need configured external services. Without these, the app may degrade to demo or fallback behavior.

3. Database provider consistency

The actual Prisma datasource is SQL Server, while some docs and example config files may imply a different setup. This is a known configuration mismatch risk.

4. Hybrid Node/Python operational complexity

The app depends on both Next.js and Python service components. Operational work must cover both, or key flows will fail.

5. AI quality dependency

The EQS and validation flows use model-based scoring, but they are not standalone if Ollama or the AI service is missing.

6. Fact-check service dependency

Web and historical fact-check routes are only as good as the available providers and their API credentials.

7. Demo/fallback patterns are project-level behavior

Some modules intentionally degrade to demo values or fallback logic; this is useful for local development, but it can mask missing production configuration.

Suggested mitigation

- confirm runtime expectations in deployment config
- validate env files against actual code requirements
- ensure database schema matches target DB provider
- verify model and AI service health before relying on scoring features
- treat local/demo fallback logic as development resilience, not a production guarantee

Status

The application is feature-rich and actively implemented, but it should be treated as a service-dependent sports platform rather than a simple static app.

DEVELOPER HANDOVER

17-DEVELOPER-HANDOVER

Handover summary

This project is a Next.js-based cricket platform with a large social/editorial layer, AI scoring, contest logic, and match intelligence features. It is not a limited demo; it is a serious, multi-domain application.

First steps for a new developer

1. Confirm the environment variables from the env examples 2. Validate the Prisma connection string and DB provider 3. Start required external services, especially Ollama and any Python AI service 4. Check SportMonks credentials for live match data 5. Run the app locally with the normal Next.js dev command

Recommended local startup command

Use the root package script:

```
npm run dev
```

This is the standard local Next.js development command based on the package manifest in the project root.

Critical runtime fact

The project is not implemented as Edge runtime for the main EQS route. The route explicitly declares Node.js runtime:

```
export const runtime = "nodejs";
```

This should be treated as the authoritative runtime behavior for that route and should not be assumed to be Edge unless a specific route or deployment config changes it.

Most important folders

- src/app — pages and route handlers
- src/lib — shared logic and integrations
- src/components — UI building blocks
- prisma/schema.prisma — schema of record
- ai_service — Python AI backend
- insights_service — Python insights service
- docs — deployment and architecture notes

High-priority validation checklist

- DB connectivity
- auth provider setup
- admin role and login flow
- Ollama model reachability
- SportMonks API access

- Deepgram/voice path if relevant
- fact-check provider access

One caution

The app has many features, but many of them are intentionally service-dependent. Development success depends on a fully configured environment, not just on the frontend code.

Final recommendation

Treat the codebase as a production-oriented platform requiring both application and infrastructure configuration. Start by validating the environment before debugging deeper business logic.